CNS PYTHON PRACTICAL RECORD

Experiments 1–13 | Easy-to-study format

Format: Aim • Algorithm • Program • Output • Result

Note: *Install the required library only for the experiments that mention it. DES/AES encrypted values and HMAC values may be shown as variable or sample values where applicable.*

1. Caesar Cipher

AIM

To implement a Python program to encrypt a message using the Caesar Cipher by shifting each alphabet by a given number of positions.

ALGORITHM

1. Start the program.
2. Get the message and shift value from the user.
3. Read each character in the message.
4. If it is uppercase or lowercase, shift it within the alphabet using modulo 26.
5. Keep spaces and special characters unchanged.
6. Display the encrypted message and stop.

PROGRAM

```
text = input("Message: ")
shift = int(input("Shift: "))
result = ""

for ch in text:
    if ch.isupper():
        result += chr((ord(ch)-65+shift)%26+65)
    elif ch.islower():
        result += chr((ord(ch)-97+shift)%26+97)
    else:
        result += ch

print("Encrypted:", result)
```

OUTPUT

```
Message: HELLO
Shift: 3
Encrypted: KHOOR
```

RESULT

Thus, the Caesar Cipher program was executed successfully and the plaintext was encrypted using the given shift value.

2. Monoalphabetic Cipher

AIM

To implement a Python program to encrypt a message by replacing each alphabet with a unique letter from a substitution key.

ALGORITHM

1. Start the program.
2. Define the original alphabet and a 26-letter substitution key.
3. Get the message from the user.
4. Read each character and find its position in the original alphabet.
5. Replace it with the character at the same position in the key.
6. Preserve the letter case and other characters.
7. Display the encrypted message and stop.

PROGRAM

```
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
key = "QWERTYUIOPASDFGHJKLZXCVBNM"
text = input("Message: ")
result = ""

for ch in text:
    if ch.isalpha():
        x = key[alphabet.index(ch.upper())]
        result += x if ch.isupper() else x.lower()
    else:
        result += ch

print("Encrypted:", result)
```

OUTPUT

```
Message: HELLO
Encrypted: ITSSG
```

RESULT

Thus, the Monoalphabetic Cipher program was executed successfully and the message was encrypted using the substitution key.

3. Playfair Cipher

AIM

To implement a Python program to encrypt plaintext using the Playfair Cipher with a 5×5 key matrix.

ALGORITHM

1. Start the program and define the key and plaintext.
2. Convert J to I and prepare a 5×5 matrix without repeated letters.
3. Divide the plaintext into pairs; insert X between repeated letters or at the end when needed.
4. Find the positions of both letters in the matrix.
5. If letters are in the same row, take the letters to their right.
6. If letters are in the same column, take the letters below them.
7. Otherwise, take the letters at the rectangle corners.
8. Display the encrypted text and stop.

PROGRAM

```
key = "MONARCHY"
text = "INSTRUMENTS".replace("J", "I")

letters = []
for ch in key + "ABCDEFGHIKLMNOPQRSTUVWXYZ":
    if ch not in letters:
        letters.append(ch)

m = [letters[i:i+5] for i in range(0, 25, 5)]
pairs, i = [], 0

while i < len(text):
    a = text[i]
    if i+1 == len(text) or text[i] == text[i+1]:
        b, i = "X", i+1
    else:
        b, i = text[i+1], i+2
    pairs.append((a, b))

def pos(ch):
    for r in range(5):
        for c in range(5):
            if m[r][c] == ch:
                return r, c

out = ""
for a, b in pairs:
    r1, c1 = pos(a)
    r2, c2 = pos(b)
    if r1 == r2:
        out += m[r1][(c1+1)%5] + m[r2][(c2+1)%5]
    elif c1 == c2:
        out += m[(r1+1)%5][c1] + m[(r2+1)%5][c2]
    else:
        out += m[r1][c2] + m[r2][c1]

print("Encrypted:", out)
```

OUTPUT

```
Encrypted: GATLMZCLRQXA
```

RESULT

Thus, the Playfair Cipher program was executed successfully and the plaintext was encrypted using a 5×5 key matrix.

4. Polyalphabetic (Vigenere) Cipher

AIM

To implement a Python program to encrypt plaintext using a repeating keyword and multiple substitution alphabets.

ALGORITHM

1. Start the program.
2. Get the plaintext and keyword from the user.
3. Convert both to uppercase.
4. Repeat the keyword to match the plaintext length.
5. Add the numerical values of each plaintext letter and key letter.
6. Apply modulo 26 and convert the result into a letter.
7. Display the encrypted text and stop.

PROGRAM

```
text = input("Plaintext: ").upper()
key = input("Key: ").upper()
out = ""

for i, ch in enumerate(c for c in text if c.isalpha()):
    k = key[i % len(key)]
    out += chr((ord(ch)-65 + ord(k)-65)%26 + 65)

print("Encrypted:", out)
```

OUTPUT

```
Plaintext: ATTACKATDAWN
Key: LEMON
Encrypted: LXFOPVEFRNHR
```

RESULT

Thus, the Polyalphabetic Cipher program was executed successfully and the plaintext was encrypted using the repeating keyword.

5. Hill Cipher (2×2)

AIM

To implement a Python program to encrypt plaintext using the Hill Cipher with a 2×2 key matrix.

ALGORITHM

1. Start the program and define the 2×2 key matrix.
2. Get the plaintext and convert it to uppercase.
3. Remove spaces and add X if the number of letters is odd.
4. Divide the plaintext into pairs of letters.
5. Convert each letter into a number from 0 to 25.
6. Multiply each pair by the key matrix and apply modulo 26.
7. Convert the numbers back to letters and display the ciphertext.

PROGRAM

```
key = [[3, 3], [2, 5]]
text = input("Plaintext: ").upper().replace(" ", "")

if len(text) % 2:
    text += "X"

out = ""
for i in range(0, len(text), 2):
    a = ord(text[i]) - 65
    b = ord(text[i+1]) - 65
    out += chr((3*a + 3*b) % 26 + 65)
    out += chr((2*a + 5*b) % 26 + 65)

print("Encrypted:", out)
```

OUTPUT

```
Plaintext: HELP
Encrypted: HIAT
```

RESULT

Thus, the Hill Cipher program was executed successfully and the plaintext was encrypted using matrix multiplication modulo 26.

6. DES Encryption and Decryption

AIM

To implement the DES algorithm in Python to encrypt plaintext and decrypt it back using the same secret key.

ALGORITHM

1. Start the program and import the DES and padding modules.
2. Generate an 8-byte secret key.
3. Get the message and convert it into bytes.
4. Create a DES cipher in ECB mode and pad the message.
5. Encrypt the padded message.
6. Decrypt the ciphertext using the same key and remove padding.
7. Display the encrypted data and decrypted message.

PROGRAM

```
# Install: python -m pip install pycryptodome
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

key = get_random_bytes(8)
msg = input("Message: ").encode()

cipher = DES.new(key, DES.MODE_ECB)
encrypted = cipher.encrypt(pad(msg, 8))

decrypted = unpad(
    DES.new(key, DES.MODE_ECB).decrypt(encrypted), 8
)

print("Encrypted (hex):", encrypted.hex())
print("Decrypted:", decrypted.decode())
```

OUTPUT

```
Message: HELLO
Encrypted (hex): [random hexadecimal value]
Decrypted: HELLO

Note: The encrypted value changes each time because a new key is generated.
```

RESULT

Thus, the DES program was executed successfully. The message was encrypted and then decrypted to obtain the original plaintext.

7. AES Encryption and Decryption

AIM

To implement the AES algorithm in Python to encrypt plaintext and decrypt it using the same secret key.

ALGORITHM

1. Start the program and import the AES and padding modules.
2. Generate a 16-byte secret key.
3. Get the message and convert it into bytes.
4. Create an AES cipher in ECB mode and pad the message.
5. Encrypt the padded message.
6. Decrypt the ciphertext using the same key and remove padding.
7. Display the encrypted data and decrypted message.

PROGRAM

```
# Install: python -m pip install pycryptodome
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

key = get_random_bytes(16)
msg = input("Message: ").encode()

cipher = AES.new(key, AES.MODE_ECB)
encrypted = cipher.encrypt(pad(msg, 16))

decrypted = unpad(
    AES.new(key, AES.MODE_ECB).decrypt(encrypted), 16
)

print("Encrypted (hex):", encrypted.hex())
print("Decrypted:", decrypted.decode())
```

OUTPUT

```
Message: HELLO
Encrypted (hex): [random hexadecimal value]
Decrypted: HELLO

Note: The encrypted value changes each time because a new key is generated.
```

RESULT

Thus, the AES program was executed successfully. The message was encrypted and then decrypted to obtain the original plaintext.

8. Diffie–Hellman Key Exchange

AIM

To implement the Diffie–Hellman key exchange in Python so that two parties can generate the same shared secret key.

ALGORITHM

1. Start the program.
2. Choose public prime number p and primitive root g.
3. Alice and Bob choose their private keys a and b.
4. Calculate their public keys A = g■ mod p and B = g■ mod p.
5. Exchange the public keys.
6. Calculate the shared secret at both sides.
7. Display both shared keys and check whether they are equal.

PROGRAM

```
p, g = 23, 5
a, b = 6, 15

A = pow(g, a, p)
B = pow(g, b, p)

s1 = pow(B, a, p)
s2 = pow(A, b, p)

print("Alice public key:", A)
print("Bob public key:", B)
print("Alice shared key:", s1)
print("Bob shared key:", s2)
```

OUTPUT

```
Alice public key: 8
Bob public key: 19
Alice shared key: 2
Bob shared key: 2
```

RESULT

Thus, the Diffie–Hellman key exchange program was executed successfully and both parties generated the same shared secret key.

9. RSA Digital Signature

AIM

To implement a basic RSA digital signature in Python and verify the signature using the public key.

ALGORITHM

1. Start the program and choose two prime numbers p and q.
2. Calculate n = p × q and $\phi(n) = (p–1) \times (q–1)$.
3. Choose public exponent e and calculate private exponent d.
4. Generate the signature using the private key.
5. Verify the signature using the public key.
6. Compare the verified message with the original message.
7. Display whether the signature is valid.

PROGRAM

```
p, q, e, message = 3, 11, 3, 4

n = p * q
phi = (p-1) * (q-1)
d = pow(e, -1, phi)

signature = pow(message, d, n)
verified = pow(signature, e, n)

print("Signature:", signature)
print("Verified message:", verified)
print("Valid:", verified == message)
```

OUTPUT

```
Signature: 4
Verified message: 4
Valid: True
```

RESULT

Thus, the RSA digital signature was generated using the private key and successfully verified using the public key.

10. SHA-256 Hashing

AIM

To implement a Python program to generate a SHA-256 hash value for a given input string.

ALGORITHM

1. Start the program and import the hashlib module.
2. Get the input text from the user.
3. Convert the text into bytes using encode().
4. Generate the hash using the SHA-256 algorithm.
5. Convert the hash into a hexadecimal string.
6. Display the hash value and stop.

PROGRAM

```
import hashlib

text = input("Enter text: ")
value = hashlib.sha256(text.encode()).hexdigest()

print("SHA-256:", value)
```

OUTPUT

```
Enter text: abc
SHA-256: ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
```

RESULT

Thus, the SHA-256 program was executed successfully and generated the hash value for the given input text.

11. HMAC Message Authentication

AIM

To implement HMAC in Python to check the integrity and authenticity of a message using a secret key and SHA-256.

ALGORITHM

1. Start the program and import hmac and hashlib modules.
2. Define a secret key and message.
3. Generate an HMAC value using SHA-256.
4. Generate the HMAC again for verification.
5. Compare both values using compare_digest().
6. Display the HMAC and verification result.

PROGRAM

```
import hmac
import hashlib

key = b"secret"
message = b"Hello"

tag = hmac.new(key, message, hashlib.sha256).hexdigest()
check = hmac.new(key, message, hashlib.sha256).hexdigest()

print("HMAC:", tag)
print("Verified:", hmac.compare_digest(tag, check))
```

OUTPUT

```
HMAC: [64-character hexadecimal value]
Verified: True

Note: The HMAC value is a fixed 64-character hexadecimal value for this key and message.
```

RESULT

Thus, the HMAC program was executed successfully and the message authentication value was verified.

12. Kerberos Authentication Simulation

AIM

To simulate a simple Kerberos-style user authentication process in Python.

ALGORITHM

1. Start the program.
2. Store the valid username and password.
3. Get the username and password from the user.
4. Check whether the entered details match the stored details.
5. If they match, display authentication successful and ticket granted.
6. Otherwise, display authentication failed.
7. Stop the program.

PROGRAM

```
users = {"alice": "pass123"}

name = input("Username: ")
password = input("Password: ")

if name in users and users[name] == password:
    print("Authentication successful")
    print("Ticket granted")
else:
    print("Authentication failed")
```

OUTPUT

```
Username: alice
Password: pass123
Authentication successful
Ticket granted
```

RESULT

Thus, the Kerberos authentication simulation was executed successfully and access was granted for valid login details.

13. X.509 Certificate Authentication

AIM

To simulate X.509 certificate authentication in Python by creating a certificate and verifying its digital signature.

ALGORITHM

1. Start the program and import the cryptography modules.
2. Generate an RSA private key.
3. Create the certificate subject and issuer.
4. Generate a self-signed X.509 certificate.
5. Verify the certificate signature using the public key.
6. Display the certificate creation and signature verification results.
7. Stop the program.

PROGRAM

```
# Install: python -m pip install cryptography
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from datetime import datetime, timezone, timedelta

key = rsa.generate_private_key(
    public_exponent=65537, key_size=2048
)

name = x509.Name([
    x509.NameAttribute(NameOID.COMMON_NAME, "Lab User")
])

now = datetime.now(timezone.utc)

cert = (
    x509.CertificateBuilder()
    .subject_name(name).issuer_name(name)
    .public_key(key.public_key())
    .serial_number(x509.random_serial_number())
    .not_valid_before(now-timedelta(minutes=1))
    .not_valid_after(now+timedelta(days=30))
    .sign(key, hashes.SHA256())
)

key.public_key().verify(
    cert.signature,
    cert.tbs_certificate_bytes,
    padding.PKCS1v15(),
    cert.signature_hash_algorithm
)

print("Certificate created")
print("Signature verified")
```

OUTPUT

```
Certificate created
Signature verified
```

RESULT

Thus, the X.509 certificate was created successfully and its digital signature was verified using the public key.

CNS Practical Lab – Experiments 14 to 22

Continuation of the 22-experiment practical record. Each experiment is arranged as Aim, Algorithm, Program, Sample Output and Result.

Experiment 14: SSL/TLS Secure Communication (Local Simulation)

AIM

To demonstrate message communication between a client and a server using Python sockets.

ALGORITHM

1. Import socket and threading modules.
2. Create a server socket and bind it to the local host and port 5001.
3. Listen for a client connection and accept it.
4. Send a message from the server to the client.
5. Connect the client, receive the message, and display it.
6. Close both sockets and stop the thread.

PROGRAM

```
import socket, threading
def server():
    s = socket.socket()
    s.bind(("127.0.0.1", 5001)); s.listen(1)
    conn, _ = s.accept()
    conn.sendall(b"Hello secure client")
    conn.close(); s.close()
t = threading.Thread(target=server); t.start()
c = socket.socket(); c.connect(("127.0.0.1", 5001))
print("Server message:", c.recv(1024).decode())
c.close(); t.join()
```

SAMPLE OUTPUT

```
Server message: Hello secure client
(Note: this is a socket demo, not a real TLS-encrypted connection.)
```

RESULT

Thus, local client-server communication was demonstrated successfully using Python sockets. This short program does not implement TLS encryption.

Experiment 15: PGP Secure Email (Base64 Teaching Simulation)

AIM

To demonstrate encoding and decoding of an email message using Base64.

ALGORITHM

1. Import the Base64 module.
2. Read the email message from the user.
3. Convert the message into bytes and encode it using Base64.
4. Decode the encoded data back into the original message.
5. Display the encoded and decoded messages.

PROGRAM

```
import base64
message = input("Email message: ")
encoded = base64.b64encode(message.encode()).decode()
decoded = base64.b64decode(encoded).decode()
print("Encoded:", encoded)
print("Decoded:", decoded)
```

SAMPLE OUTPUT

```
Email message: Hello email
Encoded: SGVsbG8gZW1haWw=
Decoded: Hello email
```

RESULT

Thus, the email text was encoded and decoded successfully. Base64 is only an encoding method, not encryption or a complete PGP implementation.

Experiment 16: S/MIME-like Secure Email (AES-GCM + RSA)

AIM

To demonstrate email confidentiality by encrypting a message with AES-GCM and protecting the AES key with RSA.

ALGORITHM

1. Import the required cryptography modules.
2. Generate an RSA private key and obtain its public key.
3. Read the email message.
4. Generate an AES key and a random nonce.
5. Encrypt the email using AES-GCM.
6. Encrypt (wrap) the AES key using the RSA public key.
7. Decrypt the wrapped AES key using the RSA private key.
8. Decrypt and display the original email using AES-GCM.

PROGRAM

```
# Install: python -m pip install cryptography
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
private = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public = private.public_key()
msg = input("Email: ").encode()
key = AESGCM.generate_key(bit_length=128)
nonce = os.urandom(12)
encrypted = AESGCM(key).encrypt(nonce, msg, None)
wrapped = public.encrypt(key, padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
                algorithm=hashes.SHA256(), label=None))
key2 = private.decrypt(wrapped, padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
                 algorithm=hashes.SHA256(), label=None))
print("Decrypted email:", AESGCM(key2).decrypt(nonce, encrypted, None).decode())
```

SAMPLE OUTPUT

```
Email: Hello
Decrypted email: Hello
```

RESULT

Thus, the email message was encrypted and then recovered successfully using AES-GCM and RSA key wrapping.

Experiment 17: Firewall and IDS/IPS Simulation

AIM

To simulate basic firewall and intrusion detection/prevention rules for network traffic.

ALGORITHM

1. Define a list of blocked IP addresses and allowed ports.

2. Get the source IP, port number, and failed-login count.

3. Check whether the IP is blocked, the port is not allowed, or failed logins are five or more.

4. If any condition is true, display a blocked-traffic alert.

5. Otherwise, display that traffic is allowed.

PROGRAM

```
blocked = ["192.168.1.100"]
allowed_ports = [80, 443]
ip = input("Source IP: ")
port = int(input("Port: "))
attempts = int(input("Failed logins: "))
if ip in blocked or port not in allowed_ports or attempts >= 5:
    print("ALERT: Traffic blocked")
else:
    print("Traffic allowed")
```

SAMPLE OUTPUT

```
Source IP: 192.168.1.100
Port: 80
Failed logins: 1
ALERT: Traffic blocked
```

RESULT

Thus, the firewall/IDS-IPS rule simulation identified the blocked source and denied the traffic.

Experiment 18: Zero Trust Authentication

AIM

To demonstrate access verification using username, password, device identity, and resource permission.

ALGORITHM

1. Define the user credentials and role.
2. Read the username, password, and device name.
3. Verify the username, password, and approved device.
4. If verification succeeds, ask for the requested resource.
5. Compare the requested resource with the user's role.
6. Grant access only when all checks succeed; otherwise deny access.

PROGRAM

```
users = {"alice": ("1234", "admin")}
name = input("Username: ")
pwd = input("Password: ")
device = input("Device: ")
if name in users and users[name][0] == pwd and device == "laptop1":
    resource = input("Resource (admin/user): ")
    if users[name][1] == resource:
        print("Identity verified – Access GRANTED")
    else:
        print("Access DENIED: permission mismatch")
else:
    print("Access DENIED: verification failed")
```

SAMPLE OUTPUT

```
Username: alice
Password: 1234
Device: laptop1
Resource (admin/user): admin
Identity verified – Access GRANTED
```

RESULT

Thus, the Zero Trust demonstration granted access after checking the user, password, device, and resource permission.

Experiment 19: Post-Quantum Kyber + AES-GCM

AIM

To demonstrate a post-quantum key exchange using Kyber (ML-KEM) and secure message encryption using AES-GCM.

ALGORITHM

1. Install pqcrypto and cryptography.
2. Generate Kyber public and private keys.
3. Encapsulate a shared secret using the public key.
4. Decapsulate it using the private key to obtain the receiver's secret.
5. Hash both shared secrets to derive AES keys.
6. Encrypt a message using AES-GCM and the sender's key.
7. Decrypt it using the receiver's key and compare the derived keys.

PROGRAM

```
# Install: python -m pip install pqcrypto cryptography
import hashlib, os
from pqcrypto.kem.ml_kem_768 import keygen, encaps, decaps
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
public, private = keygen()
ct, sender_secret = encaps(public)
receiver_secret = decaps(private, ct)
key1 = hashlib.sha256(sender_secret).digest()
key2 = hashlib.sha256(receiver_secret).digest()
nonce = os.urandom(12)
msg = b"Hello Kyber"
encrypted = AESGCM(key1).encrypt(nonce, msg, None)
print("Decrypted:", AESGCM(key2).decrypt(nonce, encrypted, None).decode())
print("Verified:", key1 == key2)
```

SAMPLE OUTPUT

```
Decrypted: Hello Kyber
Verified: True
```

RESULT

Thus, both sides derived the same shared key, and the encrypted message was decrypted successfully.

Experiment 20: Homomorphic Encryption using TenSEAL

AIM

To demonstrate homomorphic addition by adding encrypted values and decrypting the result.

ALGORITHM

1. Install the TenSEAL library.
2. Create a BFV encryption context.
3. Encrypt the first value and the second value as BFV vectors.
4. Add the encrypted vectors.
5. Decrypt the result.
6. Display the decrypted sum.

PROGRAM

```
# Install: python -m pip install tenseal
import tenseal as ts
ctx = ts.context(ts.SCHEME_TYPE.BFV, poly_modulus_degree=8192, plain_modulus=1032193)
a = ts.bfv_vector(ctx, [25])
b = ts.bfv_vector(ctx, [35])
result = (a + b).decrypt()[0]
print("Decrypted sum:", result)
```

SAMPLE OUTPUT

```
Decrypted sum: 60
```

RESULT

Thus, homomorphic addition was demonstrated, and the decrypted sum of 25 and 35 was 60.

Experiment 21: Secure Multi-Party Computation using MPyC

AIM

To compute the sum of three parties' private inputs using secure multi-party computation.

ALGORITHM

1. Install MPyC and save the program as smpc_demo.py.
2. Import and initialize the MPyC runtime.
3. Create a secure integer data type.
4. Each of the three parties provides its private input.
5. Compute the sum using secure arithmetic.
6. Reveal and display the combined result.
7. Run the program with three parties using the command shown below.

PROGRAM

```
# Install: python -m pip install mpyc
# Save as smpc_demo.py; run: python smpc_demo.py -M 3
from mpyc.runtime import mpc
async def main():
    secint = mpc.SecInt()
    x = mpc.input(secint(100), senders=0)
    y = mpc.input(secint(200), senders=1)
    z = mpc.input(secint(300), senders=2)
    total = await mpc.output(x + y + z)
    if mpc.pid == 0:
        print("Combined sum:", total)
mpc.run(mpc.start())
mpc.run(main())
mpc.run(mpc.shutdown())
```

SAMPLE OUTPUT

```
Combined sum: 600 (shown by party 0)
Execution: python smpc_demo.py -M 3
```

RESULT

Thus, three parties' inputs were combined using the MPyC demonstration, producing the total 600.

Experiment 22: Attribute-Based Access Control (ABE-Style Demonstration)

AIM

To demonstrate granting access to encrypted data only when a user's attributes satisfy the access policy.

ALGORITHM

1. Install PyCryptodome.
2. Generate a random AES key and define the access policy.
3. Define the user's attributes and the confidential message.
4. Encrypt the message using AES-GCM.
5. Check whether the policy attributes are included in the user's attributes.
6. If the policy is satisfied, decrypt and display the message; otherwise deny access.

PROGRAM

```
import os
from Crypto.Cipher import AES
key = os.urandom(32)
policy = {"DEPT_CSE", "ROLE_FACULTY"}
user = {"DEPT_CSE", "ROLE_FACULTY"}
msg = b"Confidential data"
cipher = AES.new(key, AES.MODE_GCM)
encrypted, tag = cipher.encrypt_and_digest(msg)
if policy.issubset(user):
    dec = AES.new(key, AES.MODE_GCM, nonce=cipher.nonce)
    print("Access granted:", dec.decrypt_and_verify(encrypted, tag).decode())
else:
    print("Access denied")
```

SAMPLE OUTPUT

```
Access granted: Confidential data
```

RESULT

Thus, the demonstration granted access because the user's attributes satisfied the defined policy. This is an ABE-style access-control simulation, not a full CP-ABE implementation.